

Full length article

MS-Workset: A novel worker misbehavior dataset for workers' behavior tracking in mobile crowdsensing environment

Ahmed Abduljawad^{ID}, Hanane Lamaazi^{ID}*, Muhusina Ismail^{ID}, Omar EL Harrouss, Saed Alrabaee, Elarbi Badidi^{ID}

College of Information Technology, UAE University, Al Ain, United Arab Emirates

ARTICLE INFO

Dataset link: <https://github.com/HananeLAMA/AZI/SUREPlus-24.git>

Keywords:

Mobile crowdsensing (MCS)
Worker misbehavior
Data collection
Data integrity
Trustworthiness
Task allocation
Sensor falsification

ABSTRACT

Mobile Crowdsensing (MCS) technologies have significant potential for effective data collection and processing. However, a major challenge facing these systems is the presence of misbehaving workers. In this paper, we identify various types of worker misbehavior and the methods by which they are carried out. We propose novel approaches and datasets to address misbehavior in isolated and compound scenarios. Our simulations of these proposed solutions first contribute to the literature by providing a dedicated dataset for simulating workers' misbehavior in MCS, addressing the lack of such datasets. Second, provide a detection analysis to validate our approach and demonstrate its effectiveness and efficiency across the three simulated scenarios, namely low-quality workers, GPS spoofing, and multi-accounts. Validation results using rule-based and machine-learning algorithms indicate that the system maintains an accuracy above 98%. Also, the system consistently removes misleading reports without disrupting the data's natural pattern. Overall, the approach can enhance the reliability of Mobile Crowdsensing platforms and reduce the risk of decisions being influenced by incorrect or corrupted information.

1. Introduction

Mobile crowdsensing (MCS) has emerged over the last decade as a new paradigm for data collection and processing (Boubiche et al., 2019). Instead of a company sending its employees out with specialized equipment to capture and sense data, crowdsensing turns every person with a mobile device, such as a smartphone, tablet, or smart glasses, into a potential worker. Initially, the system is interested in sensing a phenomenon in a specific area, so it creates a new task and allocates it to suitable workers (e.g., people within a 5 km radius of the point of interest who have the appropriate sensors in their devices).

There are two main categories of MCS systems: *participatory* and *opportunistic*. Participatory systems depend on direct user involvement (Capponi et al., 2019). The system selects workers based on predefined criteria and sends them a task notification they can accept or decline. Accepting a task requires traveling to the assigned location and using one or more sensors on the mobile device (e.g., camera, microphone, gyroscope, etc.) to record the required information. In opportunistic systems, user involvement is minimal. After initial consent, typically obtained during app installation and acceptance of the terms of service, the system collects sensor data automatically as needed. This may

include data from GPS, accelerometers, or other embedded sensors without requiring ongoing user action.

The overall MCS sensing process comprises three main steps: *task allocation*, *data collection*, and *data extraction* (Lamaazi et al. (2022a)).

Data collection is one of the most critical challenges in MCS. It is usually sensed by heterogeneous and often untrusted workers operating under varying environmental conditions. As a consequence, the MCS system must deal with various forms of misbehavior, including multi-account attacks (similar to the Sybil attack but at the platform level), in which malicious workers create multiple fake accounts to manipulate the system, submission of unreliable, inconsistent, or low-quality data, and falsification of contextual data such as location coordinates (e.g. GPS Spoofing). These misbehaviors may arise from malicious intent or unintentional factors. For example, workers can register in the system as legitimate workers with malicious intent, either to exploit system resources to earn rewards with minimal effort or to increase their reputation and earn higher incentives. However, when a worker submits inaccurate data, this does not necessarily indicate malicious intent; it can be due to faulty or poorly calibrated sensors in the worker's device producing incorrect readings, or to the worker's lack

* Corresponding author.

E-mail addresses: 700037500@uaeu.ac.ae (A. Abduljawad), lamaazi.hanane@uaeu.ac.ae (H. Lamaazi), 201990139@uaeu.ac.ae (M. Ismail), omar.elharrouss@uaeu.ac.ae (O.E. Harrouss), salrabaee@uaeu.ac.ae (S. Alrabaee), Ebadidi@uaeu.ac.ae (E. Badidi).

<https://doi.org/10.1016/j.cose.2026.105136>

Received 19 December 2025; Received in revised form 3 August 2026; Accepted 28 August 2026

Available online 5 September 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

of expertise. Thus, distinguishing between intentional attacks and unintentional mistakes is another critical challenge in MCS systems. As a consequence, these misbehaviors lead to degraded system performance and trust, ineffective or poor decision-making, and unfair incentive distributions.

To address these risks, previous research has explored different approaches, including reputation and trust models, data fusion methods, anomaly detection, and machine learning-based solutions. While these approaches demonstrated promising detection results, they are often constrained by limited ground-truth availability or a lack of suitable Real-world MCS datasets, which makes distinguishing normal from malicious behavior very challenging. Similarly, existing synthetic datasets are more simplistic and focus on either task, reputation, or data consistency, with no real-world scenarios of worker behavior. Thus, developing a structured, behavior-based dataset that enables systematic and reproducible evaluation of misbehavior detection approaches in an MCS environment is needed.

These challenges raise two main research questions:

1. How can different types of worker misbehavior in mobile crowdsensing systems be realistically simulated and represented through a structured dataset for empirical evaluation?
2. How can worker trustworthiness be identified under diverse misbehavior conditions while preserving high data quality and minimizing incorrect exclusion of trustworthy workers?

To address these research questions and the gaps in the literature, a new dedicated MCS dataset for worker-centric misbehavior, called MS-Workset, is proposed. The dataset primarily focuses on participatory MCS, in which users are actively involved in the sensing process by accepting or declining task requests, and workers may exhibit various forms of misbehavior.

In this proposed dataset, three main representative misbehavior scenarios are modeled: multi-account behavior, low-quality data submission, and location data manipulation (GPS Spoofing). The developed dataset provides a controlled environment for generating diverse adversarial scenarios and enables extensive, reproducible benchmarking of trust- and behavior-aware models.

The main contributions this work provides are:

- A controlled simulation model that captures multiple worker behaviors when operating in an MCS environment.
- Integration of three main MCS misbehavior scenarios, including identity-based attacks through multiple account usage, data-centric manipulation through falsifying or providing low-quality reporting, and context manipulation using fake GPS activity.
- A structured worker-level dataset, MS-Workset, designed to support systematic analysis of misbehavior patterns and trust evaluation in mobile crowdsensing environments.

To validate and demonstrate the utility of the proposed dataset, various detection mechanisms are used as follows:

- A rule-based detection framework that identifies misbehavior by analyzing behavioral consistency, report quality indicators, and location dynamics without relying on prior assumptions about workers.
- A trust classification mechanism that labels workers as trustworthy or untrustworthy based on observed behavior and detection outcomes, enabling informed worker selection.
- A feedback and reputation update process that prevents repeated selection of untrustworthy workers in future tasks, supporting sustained data quality over time.
- An extensive evaluation using both rule-based logic and supervised machine learning models to validate detection accuracy, stability, and fairness across all misbehavior scenarios.

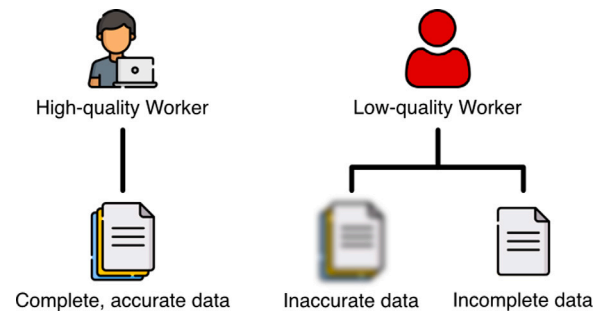


Fig. 1. A comparison of the data that high-quality workers and low-quality workers submit.

These experiments show the importance of the MS-Workset dataset, particularly for evaluating behavioral trust and data integrity scenarios, and demonstrate how it can be used to assess detection performance under controlled and diverse conditions.

2. Background and motivation

In this section, we examine scenarios involving misbehaving workers, including the different types of misbehavior, the motivations behind the behavior, and the means by which it is carried out.

2.1. Cognition-based

This category refers to misbehavior that arises from the worker's intent, awareness, or deliberate decision-making rather than from technical or environmental constraints.

2.1.1. Low-quality workers

Here, by "low-quality workers," we refer to workers who submit generally low-quality data or perform their tasks poorly, either due to malicious behavior that degrades the system performance, negligence, incompetence, or faulty sensors. For our purposes here, we make no distinction among these types, as they all have the same effect on the MCS system (see Fig. 1).

2.1.2. Adversarial workers with multiple accounts

This refers to workers who log in with multiple accounts, thereby taking on more tasks than a single person is allowed to handle and unfairly receiving more rewards in the end. This is achieved by spoofing user details and assuming a different identity, so that the system does not realize that both accounts belong to the same person, which is against the terms of many MCS systems. To limit the effect of such accounts, some systems may require certain information, such as a mobile number or ID number. However, this may have the side effect of driving away more privacy-oriented individuals (see Fig. 2).

2.2. Device-based

This section covers misbehavior arising from the device or its sensors, either due to malfunction or intentional manipulation. These issues can distort the data the system receives and may flag trustworthy workers as suspicious or allow untrustworthy workers to bypass normal task constraints.

2.2.1. Faulty device or sensor

A faulty device or a poorly calibrated sensor can unintentionally produce incorrect data, such as inaccurate GPS coordinates, irregular movement patterns, or unstable task-related readings. In such cases, a trustworthy worker may appear suspicious because the system judges only the sensor output and not the worker's intention. For example, a device with a drifting GPS chip may report sudden jumps in location

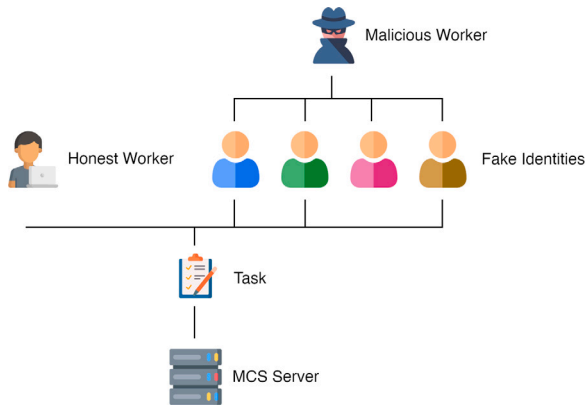


Fig. 2. Example of a worker using multiple accounts and preventing other workers from getting tasks.

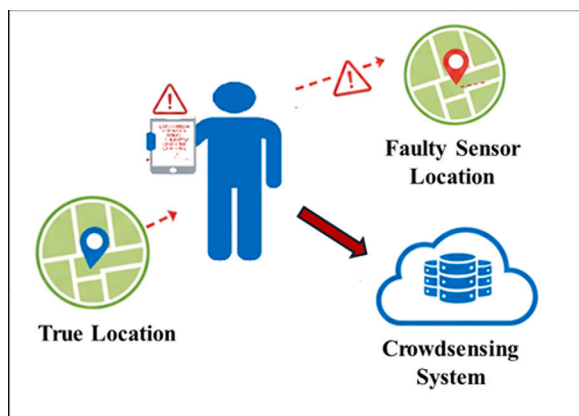


Fig. 3. Example of GPS drift caused by a faulty device sensor.

(see Fig. 3), triggering the same rules used to detect GPS spoofing. To avoid unfair misclassification, workers are responsible for keeping their device sensors up to date and ensuring proper calibration so that the system receives reliable data.

2.2.2. GPS spoofing

“Faking Sensor Data” is the act of tampering with a device’s sensors to produce values that differ from the actual ones, or of faking the existence of a sensor that does not exist on the device. For example, many apps allow users to manipulate the GPS sensor to make it appear they are at a location other than their actual one (as seen in Fig. 4). This is done to enable access to tasks that a worker would not normally have access to, which constitutes a clear act of misbehavior.

2.3. Outcomes chaser misbehavior

This section explains the misbehavior that emerges directly during task execution. These behaviors affect the system’s reliability by allowing workers to claim progress without performing the required sensing.

2.3.1. Replay attackers

“Reports Re-submission” refers to multiple devices, either owned by the same worker (as in Section 2.1.2) or not, submitting similar reports or a slightly modified one without actually sensing. This leads to degrading the quality of the MCS system, and it may be driven by many of the same motivations discussed in Section 2.1.1.

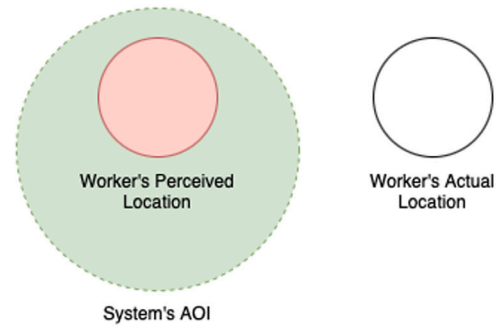


Fig. 4. Example of a worker faking GPS sensor data to get access to a system’s tasks.

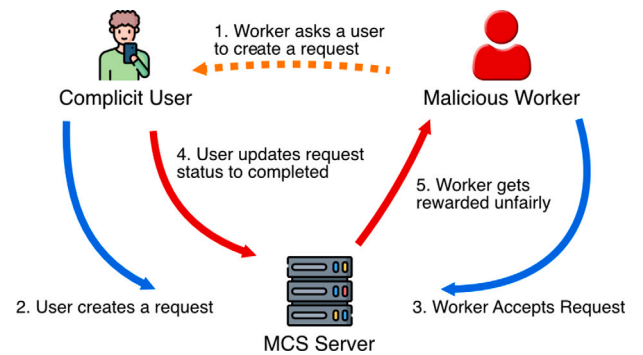


Fig. 5. Example of a worker cooperating with another user to fake tasks.

2.3.2. Target chaser

In systems where workers are required to complete a specified number of tasks each day, a careless worker who has not yet reached the required number may sign up for random tasks at the end of the day without actually completing them, thereby cheating the system. While it is straightforward to implement checks to ensure that only completed tasks count toward the worker’s daily quota, this misbehavior can paralyze the system and prevent other high-quality workers from registering for the task.

2.3.3. Collusive task falsification

As in Section 2.3.2, certain employees may create artificial requests within the system to meet their daily performance quotas. This practice involves the worker coordinating with an external party to register a nonexistent service request that the worker then formally “completes.” This process is illustrated in Fig. 5. This specific type of misconduct presents a greater detection challenge because the requesting party actively participates in the deception. The accomplice confirms task completion, even though no work was actually performed.

2.4. Existing misbehavior studies

Several studies have investigated countermeasures against malicious users in MCS systems. In this section, we review relevant research and its results and compare their approaches in Table 1.

The authors in Sucasas et al. (2020) proposed a scheme that enables users to create an unlimited number of pseudonyms and use them for different tasks. However, the scheme prevents the same worker from participating in the same task under multiple pseudonyms. Also, when a pseudonym is reported as malicious, the system allows revocation of the user’s pseudonyms, thereby preserving unlinkability while maintaining accountability. However, the proposed solution did not discuss the

Table 1
Comparative summary of existing MCS-related work.

Ref.	Misbehavior type					Detection method					Limitations
	Multiple accounts	Low-Quality data	Resubmitting reports	GPS spoofing	Pseudonyms	Device fingerprinting	Data similarity	Statistical models	ML model	Blockchain	
Sucasas et al. (2020)		✓			✓						Focuses on a single device and does not consider multiple-account attacks
Wu et al. (2019)		✓	✓		✓		✓				Identifies low-quality contributions but does not address identity abuse or fake location
Feng et al. (2019)		✓	✓				✓				Consider quality of contribution but does not provide any mechanism to detect multi-account workers or verification of location authenticity
Luo et al. (2024)		✓						✓			Focus on fraudulent reports but does not detect resubmission from multiple accounts or from falsified locations
Cui et al. (2023)			✓			✓					Mitigate unreliable data using truth discovery but cannot identify multi-account attack or location falsification
Wu et al. (2022)		✓						✓	✓		Protect the blockchain layer against the Sybil layer, but does not consider the low-quality workers or location manipulation
Arulprakash and Jebakumar (2022)		✓								✓	Focus on intrusion detection rather than worker misbehavior
Troja et al. (2021)		✓			✓		✓				verifies data integrity but does not evaluate the data source or worker legitimacy
Our Study	✓	✓	✓	✓			✓	✓			Focus on three main MCS worker misbehavior scenarios

automatic batch verification of an anomalous report and is limited to the sensing boundaries. Sensing data received from outside of the area of interest remains unverifiable and must be handled individually by the smart city units. Sensing data that falls outside reasonable bounds cannot be verified efficiently in batches and must be handled individually by the Smart City entity.

Similarly, in Wu et al. (2019), the authors propose a scheme in which users get several pseudonyms for each task they participate in; unlike in Sucasas et al. (2020), users cannot generate an infinite number of pseudonyms. After submitting reports to the task, if any reports are identified as invalid (due to high data similarity to other reports or otherwise), all pseudonyms issued to this user for this task are considered invalid, and the respective users' reputations are lowered. If a user's reputation falls below a certain threshold, the user is then removed from the system. The results show that this scheme can successfully detect and remove malicious users, provided more than 50% of the users are trustworthy.

However, discussing the scenario when malicious participants are dominant and when the invalid reports exceed 50% of the total submissions remains absent.

To combat collusive data falsification attacks, the authors in Feng et al. (2019) proposed an algorithm based on binary Min-Max clustering in which samples with high trust and low variance are considered malicious. In a simulation with 50% malicious users, this scheme showed a network trust error value of 0.28, approximately 30% lower than the baseline.

However, the proposed solution does not verify malicious workers who can behave honestly on less interesting tasks to gain trust and then launch their harmful attacks on the main target. Also, the workers' maliciousness is evaluated based on their contributions to the overall cluster, with little consideration of workers' intentions.

Although the authors in Luo et al. (2024) primarily focused on the security aspects of MCS. They included an algorithm that computes the user's trustworthiness after each sensing data report is sent and adjusts it accordingly. The authors report a detection rate of 86.5% for malicious users, who account for 40% of the system. Furthermore, with the same percentage of malicious users, reliable users are selected 91.105% of the time for tasks.

While the D2MTS improves privacy-preserving trust sharing in the MCS environment, it does not address identity abuse via Sybil or

multi-account attacks. Also, it relied mainly on historical trust evaluation without any involvement of behavioral analysis or verification of contextual data falsification, such as location spoofing.

The authors in Cui et al. (2023) addressed report re-submission using a two-step approach in which the device is fingerprinted in the first step by feeding sensor data into two LSTM layers. If the fingerprint and data similarities between multiple reports are very high, it is considered a case of forged reports. This approach was tested against the method in Wu et al. (2019) and achieved increases in trust accuracy of 8.16% to 20%. While 41% depending on the number of malicious users in the system, this approach achieved higher accuracy against systems with a majority of malicious users, with an average improvement of 14.72%.

However, they did not consider the risk of evading this detection by modifying the device characteristics, using multiple devices, or making collusive submissions. Also, the contribution did not involve the worker's misbehavior, such as faking the location or creating multiple accounts to manipulate reputation or system performance.

The authors in Wu et al. (2022) incorporated the hidden Markov model to classify users as either "High-quality users" or "Malicious users". Then, a model is trained on the classified users to pre-detect malicious users. Furthermore, a task allocation algorithm leverages historical data to maximize the data quality in future task assignments. This detection method yielded accuracies and F1 scores of approximately 0.80. The anti-malicious task-allocation method demonstrated strong performance in mitigating the effects of malicious users. Lastly, the pre-detection model accurately detected approximately 85% of malicious users, with more than 80% of the data being accurate.

However, the proposed approach primarily focuses on detecting abnormal sensing patterns and has not been tested against sophisticated attacks such as multi-account or location spoofing. Also, they did not evaluate the manipulation of behavioral patterns that leads to evading long-term detection.

The authors of Arulprakash and Jebakumar (2022) integrated blockchain technology into MCS systems to mitigate various attacks, but for our purposes, we focus on how they mitigated malicious users. This scheme leverages the blockchain's openness to enable workers to evaluate one another, thereby affecting their reputations and, in turn, their rewards. As a result, malicious users had a reputation rate below 0.3, whereas high-quality users had a reputation rate of around 0.9.

Although the proposed approach enhances data privacy and secures the reward distribution, it lacks mechanisms to address misbehavioral

patterns, such as automated submissions, fake sensing, low-quality data, or abuse of identity and contextual data.

In [Troja et al. \(2021\)](#), the authors implemented a scheme in which the workers of an MCS system are distributed into clusters. When a worker submits a value that differs substantially (either high or low) from those of other workers in the same cluster, the system rejects it as fraudulent. Additionally, the user is penalized by having their reputation reduced until it falls below a threshold, at which point the worker's input is deemed invalid. The results show a 200% improvement in fraud detection over the benchmark.

However, the proposed framework focuses primarily on fraudulent sensing reports. It does not consider other adversaries that may evade detection, such as falsifying sensing activities (e.g., Sybil attacks, multi-account attacks, or GPS Spoofing), gradually changing behavioral patterns, or exploiting compromised devices to manipulate system performance.

Prior studies have addressed different aspects of security and reliability in mobile crowdsensing systems. Truth-discovery methods have been proposed to resolve conflicting sensor data while preserving user privacy through perturbation and cloud-based processing techniques ([Zhang et al., 2020](#)). Other works have focused on protocol-level fraud resilience using formal verification to prevent Sybil and double-spending attacks ([Afzaal et al., 2023](#)), as well as on privacy-aware learning-based detection in distributed environments ([Friha et al., 2023](#)). Additional research has examined privacy and data integrity through cryptographic verification of multi-modal sensing data ([Wang et al., 2025](#)) and incentive mechanisms that balance location precision with privacy preferences ([Bedogni and Montori, 2023](#); [Song et al., 2024](#)).

However, these studies have several limitations in handling worker misbehavior. For example, the framework in [Zhang et al. \(2020\)](#) remains vulnerable to malicious workers who repeatedly inject perturbed or low-quality data or exhibit adaptive behavioral patterns to manipulate truth estimation. Similarly, the system in [Afzaal et al. \(2023\)](#) focused on transactional integrity without considering strategic attacks that may manipulate the trust metric by altering behavioral patterns or through sensing misbehavior, such as task-execution falsification or low-quality submissions. Moreover, approaches in [Friha et al. \(2023\)](#), [Wang et al. \(2025\)](#), and [Bedogni and Montori \(2023\)](#), [Song et al. \(2024\)](#) focus on preserving the data privacy using federated learning, encryption, K-anonymity, or differential privacy techniques with limited or no consideration of worker misbehavior, explicitly having a framework that jointly addresses the problem of identity abuse through multi-account attack, contextual data manipulation such as GPS spoofing, and low-quality data submission through redundant or low-quality reports.

MCS systems operate in specific ways based on task requirements. [Table 1](#) compares the literature introduced in terms of architecture, simulation parameters, and evaluation metrics.

2.5. Research gap

The growing interest in security and trust management in MCS implies the development and evaluation of efficient detection mechanisms for worker and sensing-process misbehavior. However, the existing detection mechanisms are constrained by the lack of dedicated datasets and benchmarking frameworks that illustrate the misbehavior scenarios.

2.5.1. Existing synthetic datasets used in MCS

- **CrowdsenSim:** A dedicated simulator used to generate MCS task datasets for smart cities and urban environments ([Fiandrino et al., 2017](#)). It offers several spatio-temporal features related to task requirements and user movement. However, the simulator's focus is on creating fake tasks derived from legitimate ones based on two main parameters: location and "Peak hours" ([Montori et al., 2019](#)). The simulator is not designed to model worker misbehavior; it is more task-oriented.

- **GAN model:** The Generative Adversarial Networks (GANs) model is widely used in different domains, including MCS. It can be trained on a predefined MCS dataset to generate new data indistinguishable from the original dataset ([Chen et al., 2020](#)). The GAN is not a dedicated model for simulating worker misbehavior. Still, it is more commonly used to build new datasets on top of existing ones, regardless of the dataset's purpose.

2.5.2. Most used real-time datasets in MCS

- **Geolife:** The dataset contains 182 user GPS trajectories over five years with a total distance of 1,292,951 Km. The dataset also contains users' activities such as outdoor movement, sport activities, and entertainment ([Solmaz and Zhao, 2025](#)). In MCS, the dataset has been used to evaluate worker behavior and is mostly used for the worker recruitment/selection process. However, it is not used to assess scenarios of worker misbehavior.
- **Sarwat Foursquare:** This dataset is widely used in worker selection scenarios in MCS. It is extracted from the Foursquare application API, which collects users' social networking behavior. It includes user check-ins, social connections, venues, and ratings. However, the dataset does not illustrate user misbehavior and has not been used in such scenarios in MCS ([Lamaazi et al., 2020, 2022b](#)).
- **CRAWDAD and T_Drive:** Both datasets contain mobility traces collected from taxis ([Bracciale et al., 2022](#); [Yuan et al., 2010](#)). The datasets contain features such as driver IDs, timestamps, and location coordinates. The datasets are not dedicated to MCS scenarios but are sometimes used in worker recruitment/selection or location-tracking scenarios, along with extended features. Also, the dataset is not dedicated to assessing worker misbehavior.

Overall, existing available datasets are not specifically designed to investigate MCS misbehavior scenarios. Instead, they focus on spatiotemporal patterns such as mobility traces, reputation scores, device features, and sensing data, without explicitly modeling adversarial worker behavior. As a result, no ground truth for misbehavior annotations exists, and there is a lack of attack diversity scenarios that can arise in real MCS environments.

On the other hand, existing simulation tools, such as CrowdSensim and GAN models, have been used to support MCS scenarios for task allocation, worker selection, mobility patterns, and system performance. However, these simulators were not developed to model workers' adversarial behaviors, such as multi-account activities, submission of low-quality data, and data manipulation, including GPS Spoofing.

In addition, existing datasets do not provide behavioral labels or controllable parameters. For real-world datasets, they mostly deploy existing parameters on user devices or integrate them into cars (e.g., T-Drive, CRAWDAD). Similarly, simulation-based approaches, such as CrowdSensim, used labels to assess task legitimacy without involving workers' behavior. These limit the ability to have a fair, comparative benchmarking and reproducible evaluation of the detection mechanism.

3. System model and problem formulation

This section presents the core elements of the system that support the detection of worker misbehavior. The model considers a set of spatial tasks and a group of workers who interact with these tasks through registered accounts. The following formulation establishes the common structure used to represent these interactions, which later enables the detailed analysis of each misbehavior scenario described in [Section 4](#). As it defines how worker actions and task conditions are consistently represented across the system

Table 2
Description of commonly-used notations.

Variable	Description
i, j, k	indexes for accounts, tasks and workers, respectively
I	A hyperparameter that specifies the number of iterations the simulation goes through
W, T, A	Number of workers, tasks and accounts
$\phi_0, \phi_1, \lambda_0, \lambda_1$	Minimum latitude, maximum latitude, minimum longitude, and maximum longitude, respectively
$r_0, r_{\min}, r_t, r_{t-1}$	default rating, minimum rating, current rating, and previous rating, respectively
S	Array containing data on whether a worker did a task
a_i, t_j, w_k	i th account, j th task and k th worker, respectively
d_{ij}	Distance between i th account and j th task
$d_{\max}$	Maximum acceptable distance between an account and a task
σ_{ij}^2	Variance of the reported submitted by the i account for the j th task
ω_σ	Weight of variance
L	A constant specifying the maximum possible length a worker's step can be without being considered untrustworthy.
Q_i	The Quality of report submitted by account a_i .
μ	The mean of the Quality of all submitted reports.
σ_{Quality}	The standard deviation of the Quality of all submitted reports.
f_{subtasks}	The number of finished subtasks by account a_i .
T_{subtasks}	The total number of subtasks associated with task t_j .

3.1. Key definitions

This subsection presents formal definitions of the core entities and concepts used in the proposed mobile crowdsensing security model to ensure clarity and consistency throughout the study.

- **Area of Interest (AoI):** The Area of Interest is defined as a bounded spatial region in which sensing activities are valid. Let

$$\mathcal{A} \subset \mathbb{R}^2$$

denote the AoI, where each point $a = (x, y) \in \mathcal{A}$ represents a geographic coordinate. Only sensing reports generated within $\mathcal{A}$ are accepted by the system for task evaluation.

- **Workers:** Workers are mobile participants who contribute sensing data. Let

$$\mathcal{W} = \{w_1, w_2, \dots, w_n\}$$

Be the set of all registered workers. Each worker w_i is associated with a device d_i and generates reports

$$r_{i,j} = (l_{i,j}, t_{i,j}, v_{i,j}),$$

where $l_{i,j} \in \mathcal{A}$ is the reported location, $t_{i,j}$ is the timestamp, and $v_{i,j}$ is the sensed value.

- **Untrustworthy workers:** Untrustworthy workers are workers whose reporting behavior violates system assumptions. Let

$$\mathcal{A}_w \subset \mathcal{W}$$

Represent the subset of untrustworthy workers. A worker $w_i \in \mathcal{A}_w$ produces reports that deviate from expected spatial, temporal, or content consistency constraints defined by the task.

- **Misbehavior:** Misbehavior is defined as any action that degrades data reliability. Formally, a report $r_{i,j}$ is misbehaving if

$$r_{i,j} \notin C_T,$$

where C_T is the set of valid reporting conditions for a given task T . This includes identity misuse, repeated low-quality submissions, or falsified location data.

- **Task:** A task specifies the sensing requirement assigned by the system. Let

$$T = (\mathcal{A}, \tau, \theta)$$

denote a task, where $\mathcal{A}$ is the AoI, τ is the valid time window, and θ defines data quality constraints. A report $r_{i,j}$ is valid for task T if

$$l_{i,j} \in \mathcal{A}, \quad t_{i,j} \in \tau, \quad v_{i,j} \neq \emptyset.$$

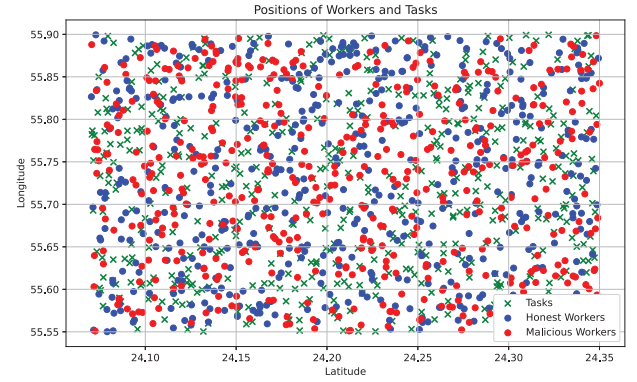


Fig. 6. A scatter plot depicting the spatial distribution of the workers within the defined geographic area of interest. All workers, including trustworthy (single-account) and malicious (multi-account) workers, are uniformly placed within the AoI to simulate a controlled MCS environment.

3.2. Worker-task execution model

The system contains a set of tasks distributed across a defined geographic area. Each task is assigned a unique identifier and positioned at coordinates (ϕ_j, λ_j) , where the latitude values lie between ϕ_0 and ϕ_1 and the longitude values lie between λ_0 and λ_1 as defined in the notation table (Table 2).

Then the workers are uniformly scattered within the same geographic region Fig. 6. To model a multi-account attack, each worker A is then assigned several accounts drawn from a geometric distribution, as illustrated in Fig. 7. This enables the simulation of diverse, heterogeneous identity-usage patterns, similar to realistic identity-manipulation scenarios in MCS, in which honest workers control a single account. In contrast, untrustworthy or malicious ones operate multiple accounts.

A worker is allowed to complete a task only when the distance between the worker's current location and the task location is within the permitted range $d_{\max}$. The distance d_{ij} between an account and a task is computed using the Haversine formula:

$$d_{ij} = 2r \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi_{ij}}{2} \right) + \cos(\phi_i) \cos(\phi_j) \sin^2 \left(\frac{\Delta\lambda_{ij}}{2} \right)} \right) \quad (1)$$

This computation ensures that task completion remains tied to physical presence and prevents workers from submitting reports on tasks located outside the allowable proximity threshold.

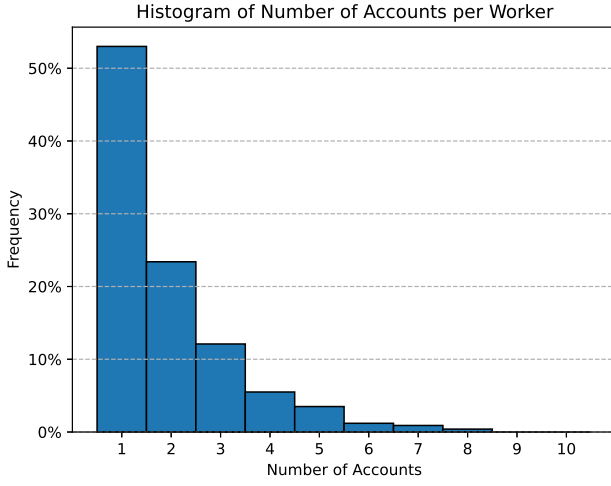


Fig. 7. Histogram depicting the frequency of usage of the owned accounts by each worker, generated using a geometric distribution. This models the multi-account misbehavior conducted by some workers during the sensing process.

3.3. Rating model

Each account begins with an initial score r_0 . After a task submission, the rating is updated based on the report variance σ_{ij}^2 , which reflects how distinct the submission is. The report variance is calculated as follows:

$$\sigma_{ij}^2 = \begin{cases} \min(1, \max(0, \text{Exp}(S))) & \text{if worker already submitted the task} \\ \min(1, \max(0, N(0.5, \text{Std}))) & \text{Otherwise} \end{cases} \quad (2)$$

Where $\text{Exp}(S)$ denotes an exponential random variable with scale S equal to 0.1 in our work, and N is the normal random variable with mean 0.5 and standard deviation (Std).

Thus, the updated rating is computed using:

$$r_t = r_{t-1} + \omega_\sigma [-(2\sigma^2 - 1)^2 + 0.1] \quad (3)$$

Where:

r_t and r_{t-1} are the current rating and previous rating, respectively. σ_{ij}^2 is the Variance of the report submitted by the i account for the j th task, and ω_σ is the Weight of variance.

To maintain the updated rating within a limited bound $[0, 1]$, it is then calculated as follows:

$$r_{\text{norm}} = \min(1, \max(0, r_t)) \quad (4)$$

If the new rating falls below the minimum rating threshold $r_{\min}$, the account is marked as suspicious. This update rule ensures that repeated or non-distinct submissions reduce the rating, while unique and consistent reports maintain it.

3.4. Report quality model

For tasks T_j that contain subtasks T_{subtasks} , the quality of report submitted by account a_i is represented by a score $Q_i \in [0, 1]$ and calculated as follows:

$$Q_i = \begin{cases} B(0.5, 0.5) & \text{if } \mathcal{W} \text{ is untrustworthy and } d_{ij} \leq D \\ \mathcal{N}(0.5, \text{SCALE}) & \text{if } \mathcal{W} \text{ is trustworthy and } d_{ij} \leq D \end{cases} \quad (5)$$

Where Untrustworthy workers submit reports that follow a probability distribution (B) to model unstable behavior, while honest workers follow a uniform distribution ($\mathcal{N}$) for more stable and centered behavior.

The system then computes a rating gradient using the Data-In Decision-Out model, as used in [Lamaazi et al. \(2022b\)](#), where features such as the average and standard deviation are extracted from

the submitted report's quality and define upper and lower thresholds accordingly. The rating gradient is then calculated as follows:

$$\text{grad}(i) = \begin{cases} \mu - Q_i & \text{if } Q_i > \mu + 0.25 \cdot \sigma_{\text{Quality}} \\ Q_i - (\mu - 0.25 \cdot \sigma_{\text{Quality}}) & \text{if } Q_i < \mu - 0.25 \cdot \sigma_{\text{Quality}} \\ \frac{1 - |Q_i - \mu|}{4} & \text{otherwise} \end{cases} \quad (6)$$

Where μ represents the mean of the Quality of all submitted reports, and σ_{Quality} is the standard deviation of the Quality of all submitted reports.

The account rating is then updated using the number of completed subtasks f_{subtasks} :

$$r_t = r_{t-1} + \text{grad}(i) - \left(0.5 - \frac{f_{\text{subtasks}}}{T_{\text{subtasks}}}\right) \quad (7)$$

Where f_{subtasks} is the number of finished subtasks by account a_i and the T_{subtasks} represent the total number of subtasks associated with task t_j .

This formulation penalizes ratings when workers submit incomplete or inconsistent reports and rewards stable performance.

4. Proposed approach

This section presents the methods used to detect the three forms of misbehavior studied in this work. Each scenario is handled by an independent rule-based model, allowing its behavior to be examined without interference from the others. Let us first outline how the dataset is created through a controlled simulation process.

4.1. Dataset creation

The dataset in this work is generated by a simulation that reflects the three misbehavior scenarios studied in the system. Each task is assigned a unique identifier and a pair of geographic coordinates. Each account receives an initial location. The path and behavior of each account follow the rules of the simulated scenario. This creates a controlled setting that enables a clear examination of misuse patterns and their influence on task interactions.

In the multi-account scenario, several accounts may belong to a single worker, and the system records the completion of repeated tasks under different identities. In the low-quality scenario, accounts produce reports with varying quality values, allowing the model to capture unstable submission behavior. In the GPS-spoofing scenario, accounts exhibit unrealistic jumps in location, as captured in their spatial traces. Each interaction between an account and a task produces a structured record containing the account ID, task ID, location coordinates, and a behavior-specific indicator such as rating, variance, or distance moved. These records constitute the CSV outputs of Phase 1 (as shown in [Fig. 8](#)) and serve as input to Phase 2 (as shown in [Fig. 9](#)) for validation, where the rule-based modules analyze the reports and identify accounts that violate the behavioral rules.

A common data structure is maintained across all scenarios, consisting of: Account ID, Task ID, Latitude, Longitude, Labels, scenario-specific behavior indicators. Using the same structure for all scenarios ensures consistency across the pipeline and provides a fair comparison between the three types of misbehavior. The simulation-driven dataset design strengthens the study by allowing clear control over how each type of misbehavior is introduced and observed. Since every scenario follows explicit behavioral rules, the resulting labels remain reliable and easy to verify. This structure makes the evaluation transparent and reproducible, ensuring that the detection results can be directly linked to the behavioral logic applied in the system. The approach supports the work's main goal: providing a clear and interpretable way to analyze misbehavior patterns across the three scenarios. The next subsection explains the three misbehavior scenarios implemented in our system and how each one is modeled in the simulation.

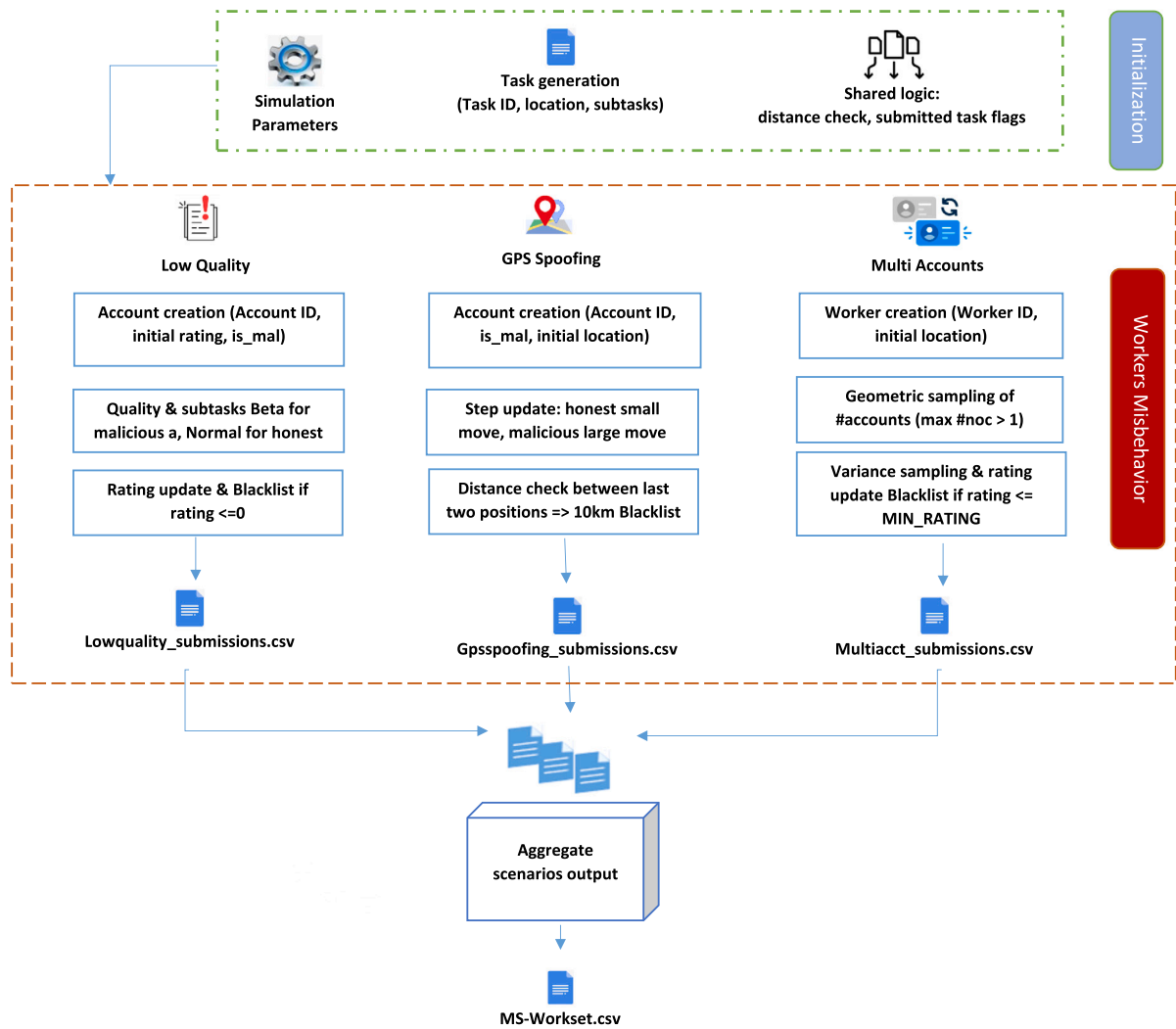


Fig. 8. Phase 1: Dataset Creation Workflow.

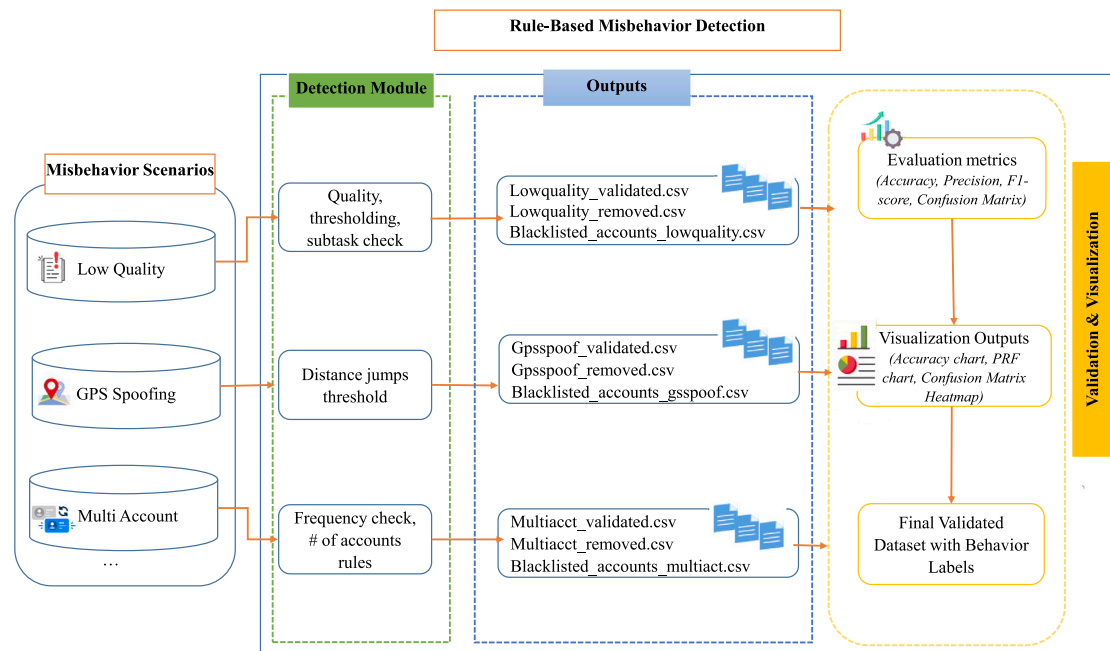


Fig. 9. Phase 2: Rule-Based Misbehavior Detection Workflow.

4.2. Scenario 1: Workers with multiple accounts

In this scenario, a worker creates multiple accounts and uses them to perform the same task repeatedly. The approach is built on the following assumptions:

1. Tasks are reusable. Any number of eligible workers can complete the same task.
2. A worker is not allowed to complete the same task more than once.
3. Tasks are location-sensitive. To complete a task, the worker must be within a distance $d_{\max}$ from the task location.
4. Tasks require a written report submitted through the account that completes the task.
5. Untrustworthy workers create more than one account and use these accounts to repeat tasks that they have already completed.
6. When repeating a task, untrustworthy workers submit identical or almost identical reports across their accounts.

The goal of this scenario is to identify accounts associated with such workers and remove them from the system. The key distinction is between a *worker* and an *account*. A worker is the human user who performs tasks. An account is the identity used by that worker to interact with the system. An untrustworthy worker can create multiple accounts and use each account as a separate identity. This allows the worker to collect more rewards while performing the same actual work.

The simulation for this scenario uses the system model described earlier. A total of T tasks are scattered over the area of interest. Their coordinates are sampled within the latitude range $[\phi_0, \phi_1]$ and the longitude range $[\lambda_0, \lambda_1]$. Then W workers are placed in the same area, and each worker is assigned several accounts A drawn from a geometric distribution. Most trustworthy workers have a single account, but each untrustworthy worker owns at least two. As a result, most accounts in the simulation are untrustworthy, even if trustworthy workers are more common. Each account starts with an initial rating r_0 . For every worker w_k , the simulation maintains a Boolean array S_k of length T . Each entry in S_k records whether the worker has already completed task t_j through any of their accounts. All entries are initially set to *False*.

During each iteration, the algorithm selects a random account a_i and shuffles the list of tasks. For each task t_j , the algorithm first checks if a_i has already completed t_j . If not, it computes the distance d_{ij} between the account and task locations using the Haversine formula in (1). If $d_{ij} < d_{\max}$, the account is eligible to complete the task. When an eligible account completes a task, the model generates a variance value σ_{ij}^2 that quantifies the similarity of this report to the worker's other reports. The variance lies in the range $[0, 1]$. A value close to 1 indicates a unique report. A value close to 0 indicates that the report is almost identical to a previous one.

If this is the first time worker w_k completes task t_j , the variance σ_{ij}^2 is drawn from a normal distribution. If w_k has already completed t_j under another account, the corresponding entry in S_k is *True*, and the variance is drawn from an exponential distribution. This models the fact that a repeated report is likely to be copied or only slightly changed. Once a new submission is accepted, the entry for t_j in S_k is set to *True*. The rating of account a_i is then updated using the rating model in (4). If the updated rating r_i falls below $r_{\min}$, the account is marked as untrustworthy. It is prevented from submitting further valid reports, and all its previous submissions are revoked.

To reflect natural movement, a small random change is applied to the worker's latitude and longitude between iterations. This keeps the spatial behavior realistic while the algorithm continues to evaluate new submissions. Algorithm 1 summarizes the steps used to identify such accounts. It combines distance checks, variance modeling, and rating updates to identify accounts that are used to perform the same task multiple times under different identities.

4.3. Scenario 2: Low-quality workers

This scenario examines workers who submit incomplete or weak reports. Each task contains a set of subtasks, and an account may complete some or all of them when submitting a report. Trustworthy accounts typically yield quality scores and subtask counts that fall within expected ranges. According to Section 2.1.1, low-quality workers are those who submit incomplete work or skip many subtasks and provide inaccurate data.

The simulation follows the same general structure used in Scenario 1 discussed in Section 4.2. Tasks are created with their subtasks, and each worker is assigned a single account, since this scenario is studied in isolation. During each iteration, a random account is selected, and the list of tasks is shuffled. If the account has not completed task t_j before and its distance to the task is within $d_{\max}$, the account becomes eligible to complete the task. Two values are then generated: the quality score Q_i and the number of completed subtasks f_{subtasks} . Trustworthy accounts draw values from both tails of a normal distribution. Low-quality accounts draw Q_i from a beta distribution with parameters $(0.5, 0.5)$, which produces values concentrated near the extremes, and f_{subtasks} from a geometric distribution, which results in fewer completed subtasks.

The quality gradient, defined earlier in (6), measures how far Q_i deviates from the expected band. The rating update rule in (7) then adjusts the account's rating based on this gradient and the number of completed subtasks. Since both equations are already provided in the problem formulation, they are not repeated here. If the updated rating falls below $r_{\min}$, the account is marked as suspicious or blacklisted. Once blacklisted, it cannot complete any further tasks, and its previous submissions are treated as invalid. This ensures that unstable or inconsistent reporting behavior is removed from the system.

Algorithm 2 presents the full procedure used in this scenario, including quality sampling, subtask sampling, gradient evaluation, and rating updates.

4.4. Scenario 3: Faking GPS data

This scenario examines workers who alter their reported location to appear closer to tasks than they actually are. Each account reports its position at every iteration, and the system evaluates whether the movement between two timestamps is realistic. Trustworthy accounts move through small steps. Untrustworthy accounts may show sudden jumps over long distances, indicating location manipulation. For each account, the system retrieves the current coordinates (ϕ_t, λ_t) and the previous coordinates $(\phi_{t-1}, \lambda_{t-1})$. The distance between the two positions is computed using the Haversine expression introduced earlier in Section 3.2 and shown in the equation. (1). This avoids repeating the formula here while keeping the notation consistent across the scenarios. If the computed distance d_t is greater than the allowed threshold L , the movement cannot be explained by normal behavior. The account is marked as suspicious and removed from the system. Its earlier submissions are also invalidated.

To preserve realistic mobility patterns, trustworthy accounts update their coordinates by a small random step at each iteration. Untrustworthy accounts use a larger step to simulate sudden jumps. Algorithm 3 outlines the full procedure, including GPS sampling, distance checks, and threshold-based identification. The operational steps for the three scenarios are given in Algorithms 1, 2, and 3.

5. Performance evaluation

In this section, a comprehensive performance evaluation is presented to validate the benefits of our proposed solutions. We implemented and simulated the solution proposed in 4, together with the simulation parameters, results, and resource consumption.

Algorithm 1: Algorithm for Detecting Workers with Multiple Accounts

```

1: Input: Number of workers  $W$ , number of tasks  $T$ , number of
   accounts  $A$ 
2: Output: List of blacklisted accounts
3: Initialize workers and tasks in the area of interest
4: Set initial worker ratings:  $r_0$ 
5: Initialize array  $S$  of size  $T$  for each worker, with all elements set
   to False
6: Scatter tasks  $t_j$  in the area with random latitudes  $\phi_j$  and
   longitudes  $\lambda_j$  between  $\phi_0, \phi_1, \lambda_0$ , and  $\lambda_1$ 
7: Scatter workers  $w_k$  in the same region, with each worker having
    $A$  accounts following a geometric distribution
8: for each iteration  $i = 1$  to  $I$  do
9:   Randomly select an account  $a_i$  from a worker's accounts
10:  Shuffle the list of tasks  $T_j$ 
11:  for each task  $t_j$  in the shuffled list do
12:    if account  $a_i$  has not completed task  $t_j$  before then
13:      Compute the distance  $d_{ij}$  between account  $a_i$  and task  $t_j$ 
        using the Haversine formula
14:      if distance  $d_{ij}$  is less than  $d_{\max}$  then
15:        Account  $a_i$  is eligible to complete task  $t_j$ 
16:        Simulate task completion and generate variance  $\sigma_{ij}^2$  based
        on the report similarity
17:        if worker  $w_k$  has previously completed  $t_j$  then
18:          Use exponential distribution to generate variance  $\sigma_{ij}^2$ 
19:        else
20:          Use normal distribution to generate variance  $\sigma_{ij}^2$ 
21:        end if
22:        Update rating of account  $a_i$  based on the variance
        Equation (4)
23:        if rating  $r_i$  is less than  $r_{\min}$  then
24:          Mark account  $a_i$  as untrustworthy
25:          Revoke all previous reports by account  $a_i$ 
26:        end if
27:      end if
28:    end if
29:  end for
30:  Randomly update the worker's position (latitude and longitude)
31: end for
32: Return the list of blacklisted accounts

```

Algorithm 2: Algorithm for Detecting Low-quality Workers

```

1: Input: Number of workers  $W$ , number of tasks  $T$ , subtasks per
   task  $T_{\text{subtasks}}$ 
2: Output: List of blacklisted accounts
3: Initialize tasks  $t_j$  with subtasks  $T_{\text{subtasks}}$  for each task
4: Initialize workers  $w_k$  with one account per worker
5: for each iteration  $i = 1$  to  $I$  do
6:   Randomly select an account  $a_i$  from a worker
7:   Shuffle the list of tasks  $T_j$ 
8:   for each task  $t_j$  in the shuffled list do
9:     if account  $a_i$  has not completed task  $t_j$  before then
10:      Compute the distance  $d_{ij}$  between account  $a_i$  and task  $t_j$ 
        using the Haversine formula
11:      if distance  $d_{ij}$  is less than  $d_{\max}$  then
12:        Account  $a_i$  is eligible to complete task  $t_j$ 
13:        Generate random floating point number  $Q_i$  for report
        quality
14:        Generate random number  $f_{\text{subtasks}}$  for number of subtasks
        completed
15:        if account  $a_i$  is trustworthy then
16:          Generate  $Q_i$  from normal distribution
17:          Generate  $f_{\text{subtasks}}$  from normal distribution
18:        else
19:          Generate  $Q_i$  from beta distribution with  $\alpha = 0.5, \beta = 0.5$ 
20:          Generate  $f_{\text{subtasks}}$  from geometric distribution
21:        end if
22:        Compute the gradient for report quality  $\text{grad}(i)$  based on
        Equation (6)
23:        Update rating  $r_i$  for account  $a_i$  using Equation (7)
24:        if rating  $r_i$  is less than  $r_{\min}$  then
25:          Mark account  $a_i$  as untrustworthy
26:          Revoke all previous reports by account  $a_i$ 
27:        end if
28:      end if
29:    end if
30:  end for
31:  Randomly update the worker's position (latitude and longitude)
32: end for
33: Return the list of blacklisted accounts

```

5.1. Simulation setup

Table 3 summarizes the evaluation parameters, such as the number of iterations I which is 100,000 for the multi-account scenario and 1000 for the low-quality scenario, number of tasks T which is set to 500, the number of workers W is 1000, $\phi_0, \phi_1, \lambda_0$ and λ_1 are 24.07, 24.35, 55.55, and 55.9 respectively, which represents the city of Al-Ain, United Arab Emirates, as seen in Fig. 10. The default rating for any new account r_0 is 0.3, and the minimum rating that an account can reach before it is removed $r_{\min}$ is 0.1. The maximum distance a worker can travel from a task and still be considered eligible $d_{\max}$ is 5 km. Lastly, the weight of the variance when updating a rating ω_σ is 0.75.

5.2. Results and analysis

This section presents the overall results of the proposed dataset creation and validation, using both rule-based and machine-learning algorithms.

5.2.1. Scenario 1: Workers with multiple accounts

Fig. 11 compares the variance reports when our algorithm is not used and when it is used. Assuming a normal distribution for generating the variances of true reports, we can evaluate the algorithm to

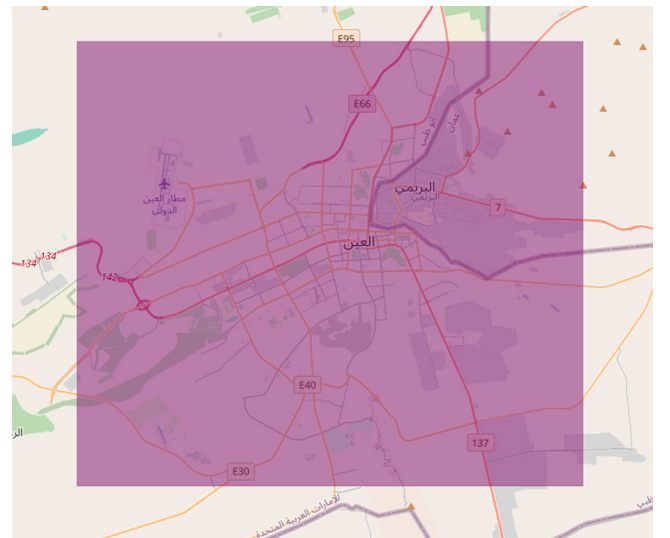


Fig. 10. Map highlighting the area of interest for our simulation, which is the city of Al-Ain in the United Arab Emirates.

Algorithm 3: Algorithm for Detecting Faked GPS Data

```

1: Input: Number of workers  $W$ , number of tasks  $T$ , subtasks per
   task  $T_{\text{subtasks}}$ 
2: Output: List of blacklisted accounts
3: Initialize tasks  $t_j$  with subtasks  $T_{\text{subtasks}}$  for each task
4: Initialize workers  $w_k$  with one account per worker
5: for each iteration  $i = 1$  to  $I$  do
6:   Randomly select an account  $a_i$  from a worker
7:   Shuffle the list of tasks  $T_j$ 
8:   for each task  $t_j$  in the shuffled list do
9:     if account  $a_i$  has not completed task  $t_j$  before then
10:      Compute the distance  $d_{ij}$  between account  $a_i$  and task  $t_j$ 
        using the Haversine formula
11:      if distance  $d_{ij}$  is less than  $d_{\text{max}}$  then
12:        Account  $a_i$  is eligible to complete task  $t_j$ 
13:        For account  $a_i$ , get current and previous GPS data  $(\phi_t, \lambda_t)$ 
        and  $(\phi_{t-1}, \lambda_{t-1})$ 
14:        Calculate the distance  $d_t$  between the current and
        previous GPS coordinates using the Haversine formula in
        Equation (1)
15:        if  $d_t > L$  then
16:          Mark account  $a_i$  as untrustworthy
17:          Remove account  $a_i$  from the system
18:        end if
19:      end if
20:    end if
21:  end for
22:  if account  $a_i$  is trustworthy then
23:    Randomly update  $w_k$ 's position by a small step
24:  else
25:    Randomly update  $w_k$ 's position by a large step
26:  end if
27: end for
28: Return: List of blacklisted accounts

```

Table 3

A table depicting values of the parameters used in our simulation.

Parameter	Value
I	100,000, 10,000
T	500, 1000, 1500
W	500, 1000, 1500
ϕ_0	24.07
ϕ_1	24.35
λ_0	55.55
λ_1	55.9
r_0	0.3
r_{min}	0.1
d_{max}	5
ω_σ	0.75
L	15

determine whether it preserves the “bell shape” distribution. As shown in the figure, the distribution shape is retained with a slight step in the left tail.

To further test how the algorithm would react to different-sized systems, we simulated worker sizes of 500, 1000, and 1500 and task sizes of 500, 1000, and 1500. The accuracy results for these runs are shown in the heatmap presented in Fig. 14(a). The heatmap presents a clear finding where the larger the system is the less accurate the algorithm is, which is to be expected considering that we are fixing the number of iterations across runs, meaning in larger systems workers are completing limited tasks than in smaller systems, as such it is harder for the algorithm to catch untrustworthy workers because the algorithm

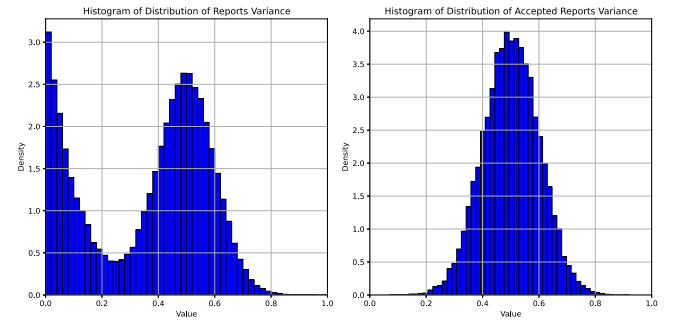


Fig. 11. Histogram of the variance of submitted reports (Left) and the variance of accepted reports when our algorithm is applied (Right).

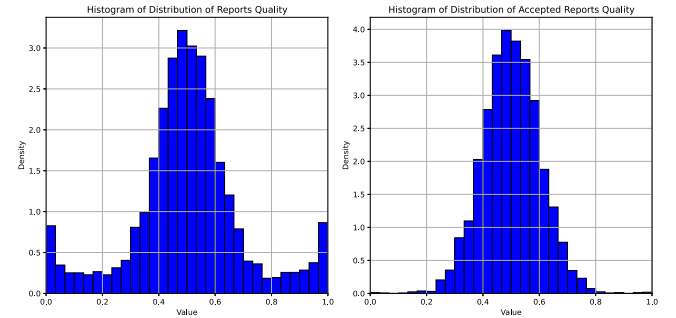


Fig. 12. Histogram of the distribution of the quality of submitted reports (Left) and accepted reports when our algorithm is applied (Right).

relies on catching workers who repeatedly submit low-variance reports and having limited reports from the same accounts means that the algorithm may miss more untrustworthy workers.

5.2.2. Scenario 2: Low-quality workers

Moving to the results and findings of low-quality worker simulations. Fig. 12 shows that our detection algorithm can remove low-quality submissions and maintain the bell curve shape of the normal distribution. In Fig. 14(b), we can see the effect of changing the size of the system on the accuracy of the algorithm. The algorithm is not change-sensitive to the number of available tasks, but accuracy decreases as the number of workers increases. However, it maintains robust accuracy regardless.

5.2.3. Scenario 3: Faking GPS data scenario

The movement paths of trustworthy and untrustworthy participants are depicted in Fig. 13, highlighting clear differences in their movement behavior. Fig. 14(c) further indicates that detection accuracy remains stable as the number of tasks increases, but is negatively affected by the increase in the number of workers.

5.2.4. Overall validation results

Table 4 reports the classification performance measured using accuracy, precision, recall, and F1 score for the three detection scenarios. The multi-account scenario achieves an accuracy of 98.6%, precision of 99.2%, and a recall of 99%, indicating stable detection performance. The low-quality scenario achieves 97.07% accuracy, whereas the fake GPS scenario achieves 98.80% accuracy, demonstrating consistent and reliable performance across different types of misbehavior. Specific runs were chosen to highlight where the algorithms excel and where they fail. For the multi-account scenario $T = 500$, $W = 1000$ were used; for the low-quality and GPS faking scenarios, $T = 1000$, $W = 1000$, and $T = 1500$, $W = 1000$ were used, respectively. By averaging the outcomes of three independent simulation runs, stable and consistent

Table 4

Rule-based detection performance and resource usage distribution across different misbehavior attack scenarios.

Resource	Metric	Multi-Account	Low-Quality	Fake GPS
–	Accuracy	98.6	97.07	98.8
	Precision	99.2	100.0	100.0
	Recall	99.0	89.37	96.03
	F1-score	99.1	94.39	97.97
CPU	TW	14.78	76.76	68.23
	UTW	61.37	16.15	24.43
RAM	TW	17.79	65.81	60.90
	UTW	73.86	19.44	21.82

TW = Trustworthy workers, and UTW = Untrustworthy workers identified during attack conditions.

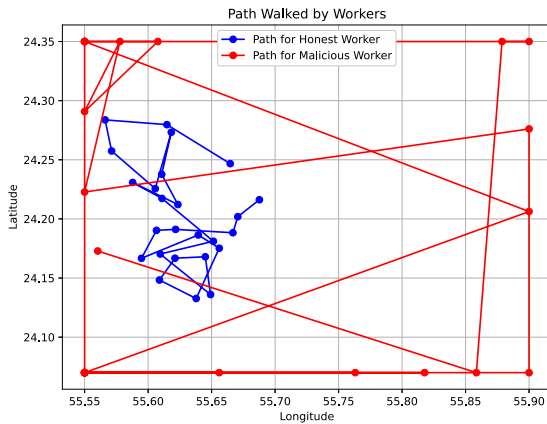


Fig. 13. Path walked by trustworthy workers (blue) and untrustworthy workers (red).

results are obtained. High accuracy is observed across all scenarios, even when untrustworthy participants form a substantial portion of the system. Precision values remain high, indicating that incorrect flagging of legitimate participants is effectively limited. This behavior is important since misclassifying trustworthy workers directly affects trust and fairness within mobile crowdsensing platforms.

5.2.5. Framework resource consumption

This section examines the computational cost and runtime behavior of the proposed detection framework under two operational settings: without attack and with attack. The analysis covers three representative misbehavior scenarios, namely low-quality reporting, multi-account activity, and fake GPS reporting. The evaluation focuses on execution time, CPU usage, and memory usage. Figs. 15(a), 15(b), and 15(c) present a comparative view of system behavior across these scenarios under both settings.

The without attack setting reflects standard platform operation. All workers follow expected behavior by maintaining a single account, submitting reports of acceptable quality, and operating from a stable geographic location without artificial movement. In this setting, all reports pass through the complete validation and scoring pipeline because no abnormal patterns are detected. This results in sustained system activity over the full execution period and provides a clear reference point for measuring computational cost under legitimate usage.

The with attack setting introduces intentional misuse that affects system behavior in different ways. Low-quality reporting requires additional checks before rejection, whereas fake GPS activity is often identified early due to location inconsistencies, thereby reducing

downstream processing. Multi-account activity increases the number of active identities and report submissions, which raises the burden on account management and tracking components. As a result, resource usage and execution time vary across scenarios depending on how each type of misuse interacts with the detection logic.

Fig. 15(a) reports the average execution time for the three detection scenarios under without attack and with attack conditions. In the low-quality scenario, without attack operation requires 279.74 s, whereas the presence of with attack activity reduces the execution time to 78.69 s. This difference arises because without attack processing applies full-quality assessment and consistency checks to all reports, whereas with attack reports are filtered earlier in the pipeline and therefore spend less time in later checks. In the multi-account scenario, the execution time increases from 42.6 s under without attack conditions to 120.1 s in the with attack case. This increase reflects the overhead of handling repeated submissions across multiple identities, thereby increasing the validation cycles. In the fake GPS scenario, without attack execution completes in 13.76 s, compared to 6.66 s when with attack behavior is present. Here, untrustworthy accounts are detected early based on location inconsistencies, and suspicious traces are excluded from further processing, thereby reducing the overall runtime. Across all scenarios, execution time varies according to the nature of the misbehavior and the point at which it is detected. These results confirm that runtime behavior is directly linked to the type and intensity of untrustworthy activity that each detection strategy handles.

The average memory usage across the three scenarios under without attack and with attack conditions is reported in Fig. 15(b). In the multi-account case, memory usage increases from 83.40% under without attack operation to 91.65% with attack behavior present. This increase occurs because multiple identities introduce additional account records, report objects, and state information that must be maintained simultaneously during processing. In the low-quality scenario, memory usage also increases from 76.90% without attack to 85.25% with attack. This behavior is linked to how both accepted and rejected reports are handled, which remain in memory until quality evaluation is complete. In the fake GPS scenario, without attack execution requires 87.40% memory usage, while with attack execution uses 82.73%. This reduction is explained by early identification of location inconsistencies, which limits the number of active movement traces retained during later processing stages.

The CPU usage distribution is depicted in Fig. 15(c). Under the without attack operation, CPU usage remains high in all scenarios, ranging from 92.00% to 96.10%, which reflects the continuous execution of the simulation loop and the full processing of incoming reports. In the low-quality scenario, CPU usage with attack remains high at 92.91%, since the system still performs extended validation and consistency checks before rejecting unreliable reports. In the fake GPS

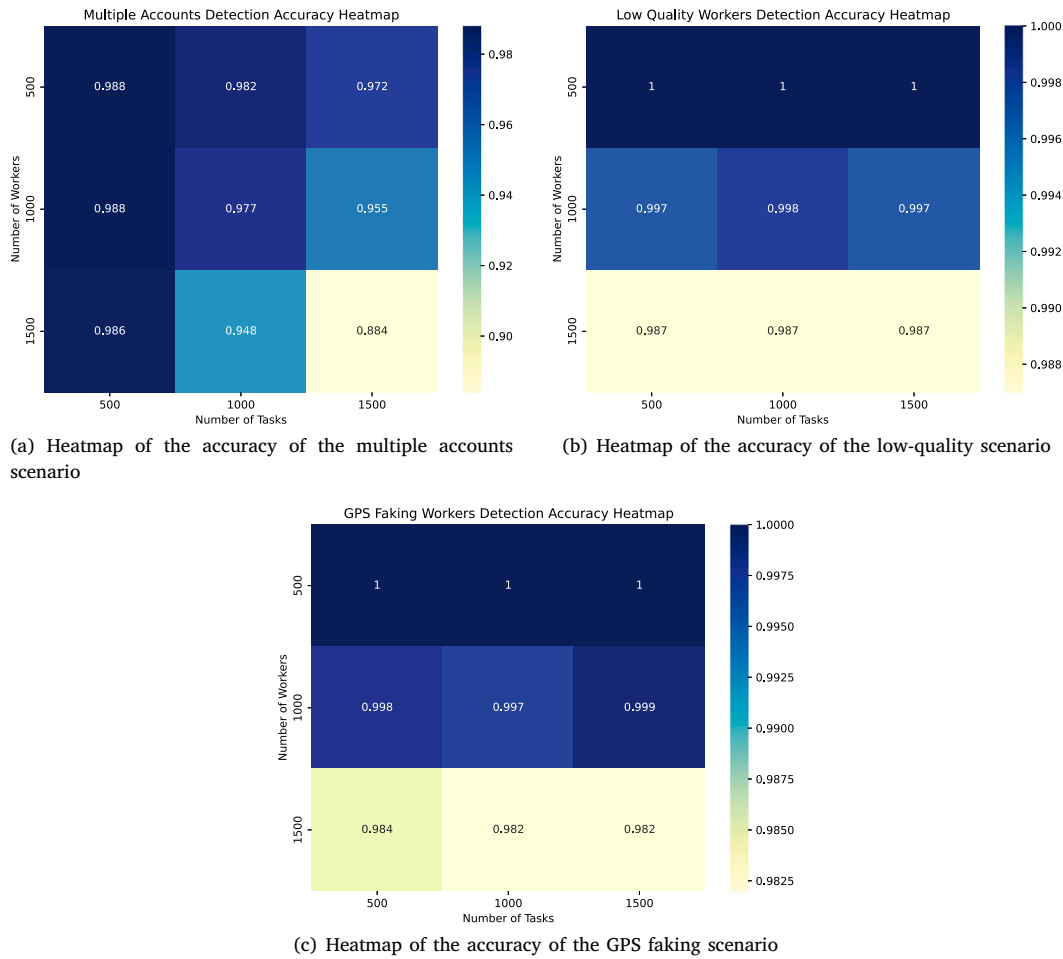


Fig. 14. Detection Accuracy Heatmap for the three scenarios validation: multiple accounts, low-quality workers, and GPS faking for different numbers of workers and tasks.

scenario, CPU usage with attack is also high at 92.66%, as location traces are continuously evaluated until suspicious behavior is confirmed. In the multi-account scenario, CPU usage decreases to 76.15% with attack. This reduction occurs because attacking accounts are detected, and the simulation completes more quickly, leading to earlier termination of repeated computation cycles. As a result, the average CPU usage appears lower even though attacking accounts trigger repeated validation steps during active phases. CPU utilization, therefore, reflects both the complexity of the detection logic and the overall runtime duration of each scenario, rather than the presence of malicious behavior alone.

Across the three scenarios, execution time remains within practical bounds while maintaining strong detection performance. The multi-account scenario records an execution time of 120.10 s with an accuracy of 98.8%, reflecting the overhead introduced by repeated submissions across multiple identities with attack. The low-quality scenario completes in 78.69 s and achieves an accuracy of 99.1%, as full report evaluation and quality assessment are performed before classification with attack. The fake GPS scenario completes fastest at 6.66 s and achieves the highest accuracy of 99.7%, since location inconsistencies allow early identification and removal of invalid traces with the attack. These results show that execution time scales with the attack type, whereas classification accuracy remains consistently high across all scenarios.

To further validate the system behavior under attack conditions, we analyze how computational resources are shared between trustworthy workers and untrustworthy workers when both coexist in the same execution environment. Table 4 reports the internal distribution of average CPU and memory usage between trustworthy and untrustworthy workers within the attack setting across all three scenarios. In the multi-account scenario, untrustworthy workers consume a higher share of CPU usage (up to 61.37%) and memory usage (up to 73.86%). This pattern results from repeated submissions across multiple identities, which require continuous validation and state maintenance. In this case, trustworthy workers use only 14.78% of the CPU and 17.79% of the memory.

In the low-quality scenario, the distribution shifts toward trustworthy workers, who consume 76.76% of CPU and 65.81% of memory. This occurs because valid reports pass through all evaluation stages and remain active in memory longer. Untrustworthy workers use a smaller share of resources at 16.15% CPU and 19.44% memory, because low-quality reports are filtered out early. A similar pattern is observed in the fake GPS scenario, where trustworthy workers account for 68.23% of CPU usage and 60.90% of memory usage. In contrast, untrustworthy workers consume 24.43% of CPU and 21.82% of memory. This occurs because location inconsistencies are detected early, thereby removing such suspicious traces. These results show that resource usage in the with attack setting depends on detection timing and processing depth, rather than on the presence of workers alone.

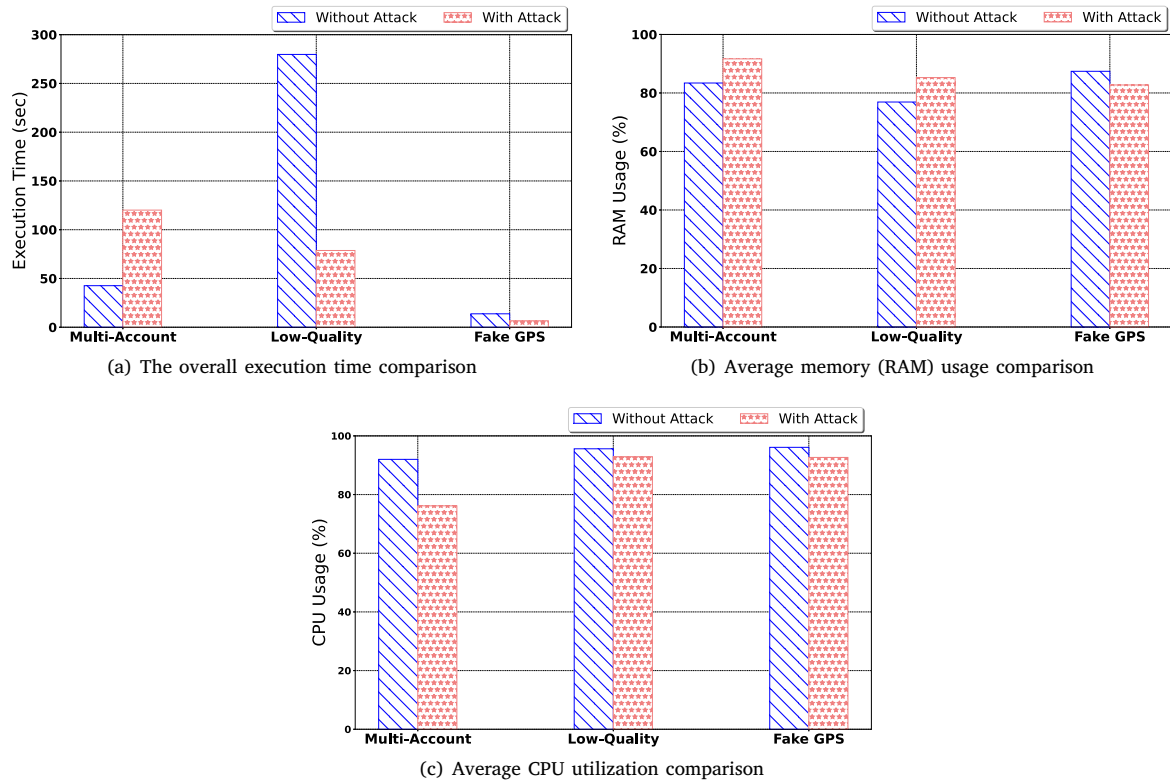


Fig. 15. Resource Consumption of the overall framework in ideal (without attack) and untrustworthy (with attack) scenarios.

5.3. Proposed framework detection validation using machine learning

To evaluate the detection capability of the proposed framework and further validate the rule-based detection outcomes, multiple supervised machine learning models were applied to each misbehavior scenario. The goal was not to replace the proposed detection logic but to confirm that the extracted behavioral features are sufficient to distinguish normal from malicious activity under realistic system conditions. Evaluation was performed using accuracy, precision, recall, and F1 score, with cross-validation (CV) used to compute average performance across folds and the standard deviation (Std) reported to reflect the stability and variability of the model results. Table 5 presents the performance of the evaluated machine learning models across the three misbehavior scenarios.

In the multi-account scenario, XGBoost and Random Forest achieve the strongest results. XGBoost records a cross-validation accuracy of 0.9662 with a standard deviation of 0.0043, indicating stable behavior across folds. Its test accuracy reaches 0.9769, supported by high precision and recall values of 0.9740 and 0.9965, respectively. The high recall confirms that repeated account usage is detected effectively, while strong precision limits the incorrect flagging of trustworthy workers. The F1 score of 0.9851 reflects a balanced outcome.

Random Forest shows similar behavior, with a cross-validation accuracy of 0.9654 and a standard deviation of 0.0048. The test accuracy of 0.9766 and the F1 score of 0.9849 confirm consistent performance. These results align with the feature structure of this scenario, where shared statistical patterns across accounts dominate.

MLP (Multi-Layer Perceptron) achieves a lower cross-validation accuracy of 0.8483 and higher variance, indicating sensitivity to data distribution. Although recall remains high, reduced precision lowers the F1 score. Gradient Boosting, SVM, and Logistic Regression show further drops in cross-validation and test accuracy. The lower scores reflect difficulty in modeling repeated behavioral patterns using linear or margin-based boundaries.

For the low-quality scenario, ensemble models perform strongly. Random Forest, XGBoost, and Gradient Boosting achieve cross-validation accuracies above 0.998 with very low standard deviations, showing high stability. All three models achieve a test accuracy of 0.9996 and a precision of 1.0000. This indicates that trustworthy reports are preserved, while unreliable submissions are consistently detected. Recall remains above 0.996, leading to F1 scores above 0.998.

SVM (Support Vector Machine) also achieves strong results, with a test accuracy of 0.9996 and stable cross-validation behavior. MLP achieves a slightly lower cross-validation accuracy of 0.9973 and a modest reduction in recall, resulting in a small drop in F1 score. Logistic Regression performs the weakest, with a test accuracy of 0.9872 and lower precision. This reflects the gradual nature of quality degradation, which is not fully captured by linear decision boundaries.

The tight alignment between cross-validation accuracy and test accuracy across models confirms that low-quality behavior produces consistent statistical signals that generalize well.

In the fake GPS scenario, Gradient Boosting achieves the strongest performance. It records a cross-validation accuracy of 0.9981 with a standard deviation of 0.0007, indicating stable learning. The test accuracy reaches 0.9988, supported by perfect precision and a high recall of 0.9909. The resulting F1 score of 0.9954 confirms effective detection of abnormal movement patterns.

Random Forest and XGBoost exhibit slightly lower recall, decreasing the F1 scores despite perfect precision. MLP and SVM record further reductions in recall and cross-validation accuracy, indicating sensitivity to spatial variability. Logistic Regression again performs the weakest, with lower recall and test accuracy, reflecting the non-linear nature of inconsistencies in geographic movement.

The separation between cross-validation and test results remains small across all models, confirming stable generalization.

Across all scenarios, Random Forest, XGBoost, and Gradient Boosting provide the strongest balance between accuracy, stability, and generalization. Their high cross-validation accuracy, together with low

Table 5
Machine learning model performance across different misbehavior detection scenarios.

Scenario	Model	CV accuracy	Std	Test accuracy	Precision	Recall	F1-score
Low-Quality	Random Forest	0.9988	0.0010	0.9996	1.0000	0.9967	0.9983
	XGBoost	0.9990	0.0010	0.9996	1.0000	0.9967	0.9983
	Gradient Boosting	0.9990	0.0008	0.9996	1.0000	0.9967	0.9983
	SVM	0.9981	0.0014	0.9996	1.0000	0.9967	0.9983
	MLP	0.9973	0.0011	0.9992	1.0000	0.9934	0.9967
	Logistic Regression	0.9880	0.0011	0.9872	0.9327	0.9636	0.9479
Fake GPS	Gradient Boosting	0.9981	0.0007	0.9988	1.0000	0.9909	0.9954
	Random Forest	0.9970	0.0018	0.9971	1.0000	0.9789	0.9893
	XGBoost	0.9970	0.0018	0.9971	1.0000	0.9789	0.9893
	MLP	0.9940	0.0029	0.9943	0.9907	0.9668	0.9786
	SVM	0.9933	0.0025	0.9938	0.9938	0.9607	0.9770
	Logistic Regression	0.9917	0.0012	0.9889	0.9872	0.9305	0.9580
Multi-Account	XGBoost	0.9662	0.0043	0.9769	0.9740	0.9965	0.9851
	Random Forest	0.9654	0.0048	0.9766	0.9744	0.9956	0.9849
	MLP	0.8483	0.0058	0.8832	0.9111	0.9393	0.9250
	Gradient Boosting	0.8415	0.0108	0.8394	0.8335	0.9878	0.9041
	SVM	0.7677	0.0020	0.7692	0.7692	0.9983	0.8689
	Logistic Regression	0.7576	0.0049	0.7511	0.7838	0.9323	0.8516

standard deviation, shows that the learned decision rules remain consistent across folds rather than being driven by random variation. Precision values remain high for these models, thereby limiting false positives and reducing the risk of incorrectly flagging trustworthy workers. Recall values are also high, confirming that malicious behavior is detected early enough to prevent impact on system outcomes.

Logistic Regression and SVM show lower performance across scenarios. Their accuracy and F1 scores decrease because the detected behaviors depend on interaction features rather than simple separable patterns. MLP improves upon these baselines but remains sensitive to dataset size and feature density, which affect stability and recall in the multi-account scenario.

The comparative results confirm that Random Forest, XGBoost, and Gradient Boosting are the most suitable choices for validating misbehavior detection in this study. Their consistent performance across accuracy, precision, recall, F1 score, and cross-validation statistics indicates that the extracted features reflect genuine behavioral patterns rather than artifacts of a specific simulation setting.

6. Conclusion and discussion

In this paper, we explored the challenges posed by worker misbehavior in MCS systems by designing a structured dataset called MS-Workset. The dataset models three representative misbehavior scenarios: multiple accounts attack, low-quality data report, and GPS spoofing. These scenarios represent the threats that affect the trustworthiness, reliability, and decision-making of the MCS system. Unlike existing synthetic or real-world data that lack ground truth or a task-based focus, MS-Workset provides a dedicated behavioral modeling platform under controllable conditions. Thus, enable a systematic and reproducible evaluation of detection mechanisms. The dataset can serve as a key benchmark for contributions on trust management, behavior-aware modeling, and anomaly detection in MCS environments.

To demonstrate the applicability and utility of the proposed dataset, we conducted extensive testing and simulations using several detection techniques, including machine learning, rule-based, and trust-based scoring. The overall results show that the dataset supports effective detection of diverse misbehavior patterns while maintaining low computational overhead. The results highlight the potential of our approach to improve the accuracy and reliability of mobile crowdsensing systems as they scale. As the current work focuses on individual misbehavior, exploring more misbehavior scenarios, such as collusive adversaries and large-scale Sybil attacks, remains an important direction for future research. Future work will focus on extending the dataset with

context-aware behavioral models and additional adversarial scenarios, as well as adopting alternative trust evaluation methods and detection techniques to improve the overall model.

CRedit authorship contribution statement

Ahmed Abduljawad: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Hanane Lamaazi:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Muhusina Ismail:** Writing – review & editing, Visualization, Software, Methodology, Formal analysis, Conceptualization. **Omar EL Harrouss:** Writing – review & editing, Visualization, Validation, Formal analysis, Conceptualization. **Saed Alrabaee:** Writing – review & editing, Visualization, Validation, Formal analysis, Conceptualization. **Elarbi Badidi:** Writing – review & editing, Visualization, Validation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Office of the Associate Provost for Research at the United Arab Emirates University (UAEU), through the Internal Research Fund, in part by SURE PLUS Grant G00004831, and in part by the Start-up Grant G00005004.

Data availability

Ms-Workset dataset is available publicly on GitHub: <https://github.com/HananeLAMAASI/SUREPlus-24.git>.

References

- Afzaal, H., Imran, M., Janjua, M.U., 2023. Formal verification of fraud-resilience in a crowdsourcing consensus protocol. *Comput. Secur.* 131, 103290. <http://dx.doi.org/10.1016/j.cose.2023.103290>.
- Arulprakash, M., Jebakumar, R., 2022. Towards developing a block chain-based advanced data security-reward model (dssecs) in mobile crowd sensing networks. *Egypt. Inform. J.* 23 (3), 405–415. <http://dx.doi.org/10.1016/j.eij.2022.03.002>.
- Bedogni, L., Montori, F., 2023. Joint privacy and data quality aware reward in opportunistic mobile crowdsensing systems. *J. Netw. Comput. Appl.* 215, 103634. <http://dx.doi.org/10.1016/j.jnca.2023.103634>.
- Boubiche, D.E., Imran, M., Maqsood, A., Shoaib, M., 2019. Mobile crowd sensing-taxonomy, applications, challenges, and solutions. *Comput. Hum. Behav.* 101, 352–370. <http://dx.doi.org/10.1016/j.chb.2018.10.028>.
- Bracciale, L., Bonola, M., Loreti, P., Bianchi, G., Amici, R., Rabuffi, A., 2022. CRAWDAD Roma/taxi. In: IEEE DataPort. <http://dx.doi.org/10.15783/C7QC7M>.
- Capponi, A., Fiandrino, C., Kantarci, B., Foschini, L., Kliavovich, D., Bouvry, P., 2019. A survey on mobile crowdsensing systems: Challenges, solutions, and opportunities. *IEEE Commun. Surv. Tutor.* 21 (3), 2419–2465. <http://dx.doi.org/10.1109/COMST.2019.2914030>.
- Chen, Z., Simsek, M., Kantarci, B., 2020. Region-aware bagging and deep learning-based fake task detection in mobile crowdsensing platforms. In: IEEE Global Communications Conference. GLOBECOM, pp. 1–7. <http://dx.doi.org/10.1109/GLOBECOM42002.2020.9322625>.
- Cui, H., Liao, J., Yu, Z., Xie, Y., Liu, X., Guo, B., 2023. Trust assessment for mobile crowdsensing via device fingerprinting. *ISA Trans.* 141, 93–102. <http://dx.doi.org/10.1016/j.isatra.2022.12.020>.
- Feng, J., Li, T., Zhai, Y., Lv, S., Zhao, F., 2019. Ensuring honest data collection against collusive CSDF attack with binary-minimax clustering analysis in mobile crowd sensing. *IEEE Access* 7, 124491–124501. <http://dx.doi.org/10.1109/ACCESS.2019.2938771>.
- Fiandrino, C., Capponi, A., Cacciatore, G., Kliavovich, D., Sorger, U., Bouvry, P., Kantarci, B., Granelli, F., Giordano, S., 2017. CrowdSenSim: a simulation platform for mobile crowdsensing in realistic urban environments. *IEEE Access* 5, 3490–3503. <http://dx.doi.org/10.1109/ACCESS.2017.2671678>.
- Friha, O., Ferrag, M.A., Benbouzid, M., Berghout, T., Kantarci, B., Choo, K.-K.R., 2023. 2DF-IDS: Decentralized and differentially private federated learning-based intrusion detection system for industrial IoT. *Comput. Secur.* 127, 103097. <http://dx.doi.org/10.1016/j.cose.2023.103097>.
- Lamaazi, H., Mizouni, R., Otrouk, H., Singh, S., Damiani, E., 2022a. Smart-3DM: Data-driven decision making using smart edge computing in a hetero-crowdsensing environment. *Future Gener. Comput. Syst.* 131, 151–165. <http://dx.doi.org/10.1016/j.future.2022.01.014>.
- Lamaazi, H., Mizouni, R., Otrouk, H., Singh, S., Damiani, E., 2022b. Trust-3dm: trustworthiness-based data-driven decision-making framework using smart edge computing for continuous sensing. *IEEE Access* 10, 133095–133108. <http://dx.doi.org/10.1109/ACCESS.2022.3231549>.
- Lamaazi, H., Mizouni, R., Singh, S., Otrouk, H., 2020. A mobile edge-based crowdsensing framework for heterogeneous IoT. *IEEE Access* 8, 207524–207536. <http://dx.doi.org/10.1109/ACCESS.2020.3038249>.
- Luo, B., Li, X., Liu, X., Guo, J., Ren, Y., Ma, S., Ma, J., 2024. D^2MTS : Enabling dependable data collection with multiple crowdsourcers trust sharing in mobile crowdsensing. *IEEE Trans. Knowl. Data Eng.* 36 (3), 927–942. <http://dx.doi.org/10.1109/TKDE.2023.3294503>.
- Montori, F., Cortesi, E., Bedogni, L., Capponi, A., Fiandrino, C., Bononi, L., 2019. CrowdSenSim 2.0: a stateful simulation platform for mobile crowdsensing in smart cities. In: Proceedings of the 22nd International ACM Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems. pp. 289–296. <http://dx.doi.org/10.1145/3345768.3355929>.
- Solmaz, S.M., Zhao, D., 2025. GeoLife dataset. In: IEEE DataPort. <http://dx.doi.org/10.21227/thw6-7y17>.
- Song, H., Shen, H., Zhao, N., He, Z., Wu, M., Xiong, W., Zhang, M., 2024. APLDP: Adaptive personalized local differential privacy data collection in mobile crowdsensing. *Comput. Secur.* 136, 103517. <http://dx.doi.org/10.1016/j.cose.2023.103517>.
- Sucasas, V., Mantas, G., Bastos, J., Damião, F., Rodriguez, J., 2020. A signature scheme with unlinkable-yet-accountable pseudonymity for privacy-preserving crowdsensing. *IEEE Trans. Mob. Comput.* 19 (4), 752–768. <http://dx.doi.org/10.1109/TMC.2019.2901463>.
- Troja, E., Yadav, N., Lin, I., Diep, N.U., 2021. Privacy-preserving crowd-sensing for dynamic spectrum access with malicious workers. *IEEE Access* 9, 24234–24248. <http://dx.doi.org/10.1109/ACCESS.2021.3057322>.
- Wang, J., Meng, F., Liu, J., He, G., Zhao, G., 2025. Multimodal data privacy protection and completeness verification method for mobile crowd sensing. *Peer-To-Peer Netw. Appl.* 18 (1), 39. <http://dx.doi.org/10.1007/s12083-024-01850-4>.
- Wu, X., Sun, Y.-E., Du, Y., Gao, G., Huang, H., Xing, X., 2022. An anti-malicious task allocation mechanism in crowdsensing systems. *Future Gener. Comput. Syst.* 127, 347–361. <http://dx.doi.org/10.1016/j.future.2021.09.016>.
- Wu, H., Wang, L., Xue, G., Tang, J., Yang, D., 2019. Enabling data trustworthiness and user privacy in mobile crowdsensing. *IEEE/ACM Trans. Netw.* 27 (6), 2294–2307. <http://dx.doi.org/10.1109/TNET.2019.2944984>.
- Yuan, J., Zheng, Y., Zhang, C., Xie, W., Xie, X., Sun, G., Huang, Y., 2010. T-drive: driving directions based on taxi trajectories. In: Proceedings of the 18th SIGSPATIAL International Conference on Advances in Geographic Information Systems. pp. 99–108. <http://dx.doi.org/10.1145/1869790.1869807>.
- Zhang, C., Xu, C., Zhu, L., Li, Y., Zhang, C., Wu, H., 2020. An efficient and privacy-preserving truth discovery scheme in crowdsensing applications. *Comput. Secur.* 97, 101848. <http://dx.doi.org/10.1016/j.cose.2020.101848>.



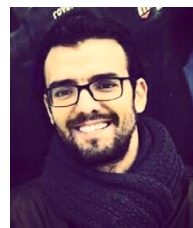
Ahmed Abduljawad received his Bachelor of Science in Computer Science from the United Arab Emirates University, graduating with honors. His research interests include human-computer interaction, computer vision, and theoretical computer science. He has contributed to projects in these areas and is particularly interested in the intersection of HCI and user experience design.



Hanane Lamaazi is currently an Assistant Professor in the College of Information Technology at the United Arab Emirates University (UAEU), where she also serves as a Faculty Fellow with the Center of Excellence in Teaching and Learning (CETL). She earned her Ph.D. in Computer Science and Networks from Moulay Ismail University in Meknes, Morocco, in 2018. She served as a Postdoctoral Fellow at the Center for Cyber-Physical Systems, Khalifa University of Science and Technology, United Arab Emirates, from 2019 to 2022. Her research interests include the Internet of Things (IoT), RPL routing protocol, edge computing, crowdsensing, cybersecurity, and risk management. She is a TPC member at different scientific conferences. She serves as an active reviewer for several highly reputable scientific journals. She was a guest speaker at several conferences.



Muhusina Ismail received her Ph.D. in Informatics and Computing from the Department of Information Systems and Security at the United Arab Emirates University in 2025. She also served as an Adjunct Faculty member in the College of Information Technology at UAEU (from 2020 to 2025). Her research focuses on web application security, machine-learning-based attack detection, Knowledge graph-driven analysis, foundation models, and explainable artificial intelligence. She has contributed to several research projects and has reviewed for academic venues in security and artificial intelligence.



Omar EL Harrouss received the Ph.D. degree from the LIAN Laboratory, Faculty of Sciences, Sidi Mohamed Ben Abdellah University, Morocco, in 2017. From February 2017 to February 2018, he was a Lecturer at the National School of Applied Sciences of Al Hoceima, Morocco. Subsequently, from 2018 to October 2021, he served as a Postdoctoral Researcher at Qatar University. In November 2021, he embarked on a new chapter by becoming an Education and Research Fellow at Mohamed 6 Polytechnic University (UM6P). His research interests include pattern recognition, image processing, and computer vision.



Saed Alrabae (Senior Member, IEEE) received the Ph.D. degree in information system engineering from Concordia University, Montreal, QC, Canada. He is currently an Associate Professor in the Department of Information Systems and Security at United Arab Emirates University (UAEU), Al Ain, United Arab Emirates. Before joining UAEU, he was a Visiting Assistant Professor in the Departments of Electrical and Computer Engineering and Computer Science at the University of New Haven in West Haven, CT, USA. He is also a Permanent Research Scientist with the Security Research Center at the Concordia Institute for Information Systems Engineering, Concordia University. He has authored or co-authored more than 50 articles in top-tier journals and at prestigious conferences. His research interests include the broad area of reverse engineering, binary authorship attribution and characterization, and malware investigation.



Elarbi Badidi received the bachelor's degree in electrical engineering and the M.Sc. degree in computer science from École Mohammadia d'Ingénieurs, Rabat, Morocco, and the Ph.D. degree in computer science from Université de Montréal, Canada. He is currently an Associate Professor with the College of Information Technology (CIT), United Arab Emirates University (UAEU). Before joining UAEU, he worked for three years as a Bioinformatics Group Leader with the Biochemistry Department at Université de Montréal. He has over 15 years of research experience in service-oriented architecture, cloud comput-

ing, and context-aware systems, with a focus on service management, service-level agreement management, quality-of-context (QoC) negotiation, QoC-based selection, and data-as-a-service provisioning. He co-edited a handbook on smart cities and published more than 80 peer-reviewed papers in reputable international journals and conferences, as well as 10 book chapters. His research interests include edge computing, cloud computing, the Internet of Things (IoT), big data, data stream processing, and data analytics. He served on the technical program committees of many international conferences and as a reviewer of several journals.